

Programming notes for Assignment 1

The Python, NumPy, SciPy and scikit-learn the notebooks use

Fall 2026 · keep it next to the notebook; nothing here answers a graded question

What this is

Assignment 1 hands you four notebooks. Each has *provided* cells, which you run as they are, and *TODO* cells, which you complete. Every *TODO* cell lists the functions it expects you to use. This handout is the reference for those functions: what each one takes, what it returns, and what its error messages mean. The mathematics behind them is in the companion handout, *Linear algebra for this course*.

It is not a solution set. Every example below runs on a toy array you can type in and check by eye; none of it is the assignment's data or the assignment's computation. If a line here looks like it could be pasted into a *TODO* cell as the answer, it is not, and the reason is usually that the assignment asks for the same idea applied to a whole matrix at once.

The one habit. Print the shape of everything. Nearly every bug in numerical code is a shape bug, and shape bugs announce themselves the moment you look.

1 Python in one page

1.1 Containers

```
xs = [3, 1, 4] # list: grows, holds anything
xs.append(1)
d = {'cat': 0, 'dog': 1} # dict: look things up by name
d['cat'] # -> 0
shape = (120, 4096) # tuple: fixed; shapes are tuples
shape[0] # -> 120
```

A NumPy array is the fourth container and the one that matters: a block of numbers of one type with a shape. Lists are for bookkeeping; arrays are for arithmetic.

1.2 Functions return, they do not print

```
def halfway(a, b):
    """The point midway between a and b."""
    return (a + b) / 2

m = halfway(np.array([0., 2.]), np.array([4., 4.])) # array([2., 3.] )
```

A function that prints its answer cannot be reused; one that returns it can. The TODO cells that ask for a function will call it later in the notebook, so it must return.

1.3 Loops, comprehensions, and pairs

```
squares = [v ** 2 for v in xs] # a loop that builds a list
for i, v in enumerate(xs): ... # index and value together
for name, value in zip(names, values): ... # two sequences in step
for layer in ('early', 'middle', 'late'): ... # loop over strings
```

Loops over *layers* or *models* are fine and expected. Loops over the *entries of an array* are usually a sign that NumPy could do it in one line; §2 shows how.

1.4 Reading an error message

Read the *last* line first: it names the error. Then read upward to find the line of *your* code (not the library's) that caused it. §6 lists the ones you will meet.

2 NumPy: arrays, shapes, and the axis argument

2.1 Shape, dtype, and one row per stimulus

```
A = np.array([[1., 2.],
              [3., 4.],
              [5., 6.]])
A.shape # (3, 2): 3 rows, 2 columns
A.ndim # 2
A.dtype # float64
```

The convention. A data matrix is **stimuli by features**: one row per image, one column per pixel, unit, or neuron. So $X[i]$ is the vector for stimulus i and $X.shape[0]$ is how many stimuli there are. The assignment's matrices are, for example, (120, 4096): 120 images, 4,096 units.

2.2 Indexing and slicing

```
A[0] # array([1., 2.]) row 0
A[:, 1] # array([2., 4., 6.]) column 1
A[:2] # rows 0 and 1
A[-1] # array([5., 6.]) last row
A[1, 0] # 3.0 row 1, column 0
```

Indices start at 0, so the last valid row of a 120-row matrix is 119. A slice is a view of the same memory: changing $A[0]$ changes A .

2.3 The axis argument

The axis you name is the axis that disappears. `A.mean(axis=0)` averages down the rows and leaves one number per column. `A.mean(axis=1)` averages across the columns and leaves one number per row.

```
A.mean(axis=0) # array([3., 4.]) shape (2,)
A.mean(axis=1) # array([1.5, 3.5, 5.5]) shape (3,)
A.mean(axis=1, keepdims=True) # array([[1.5], [3.5], [5.5]]) shape (3, 1)
A.sum(), A.max(axis=0), A.std(axis=1) # same rule for every reduction
```

`keepdims=True` keeps the reduced axis as size 1, which is what lets the result be subtracted back from A (next section).

2.4 Broadcasting

When two arrays have different shapes, NumPy stretches any size-1 dimension to match. Compare shapes *from the right*: each pair must be equal, or one of them must be 1.

```
A - A.mean(axis=0) # (3,2) - (2,) -> each column minus its own mean
# array([[ -2.,  -2.],
# [ 0.,  0.],
# [ 2.,  2.]])
A - A.mean(axis=1, keepdims=True) # (3,2) - (3,1) -> each row minus its own mean
# array([[ -0.5,  0.5],
# [ -0.5,  0.5],
# [ -0.5,  0.5]])
A - A.mean(axis=1) # (3,2) - (3,) -> ValueError: operands could not be
# broadcast together with shapes (3,2) (3,)
```

The first two do genuinely different things, and `keepdims` is what selects between them. The third fails because `(3,)` lines up against the *columns* (2), not the rows. `v[:, None]` turns a `(3,)` vector into `(3, 1)` by hand, the same effect as `keepdims`.

2.5 Boolean masks

```
labels = np.array(['a', 'b', 'a'])
mask = (labels == 'a') # array([ True, False,  True])
A[mask] # rows 0 and 2
mask.sum() # 2 (True counts as 1)
mask.mean() # 0.667: the FRACTION that is True
A[mask].mean(axis=0) # array([3., 4.]): the mean row of group 'a'
np.isin(labels, ['a', 'c']) # membership test, one boolean per entry
```

Combine masks with `&`, `|`, `~`, always with parentheses: `(labels == 'a') | (labels == 'b')`. Writing `and` or `or` between arrays raises “truth value of an array is ambiguous”. “For each group, the mean row” is the last idiom applied once per group, with `np.stack` to put the results back into one matrix.

2.6 Sorting, unique values, and the upper triangle

```
v = np.array([0.3, 0.1, 0.7, 0.2])
np.argsort(v) # array([1, 3, 0, 2]): INDICES, smallest first
np.argsort(v)[::-1] # largest first
np.unique(labels, return_counts=True) # (array(['a', 'b']), array([2, 1]))

iu = np.triu_indices(4, k=1) # upper triangle of a 4 x 4, no diagonal
iu # (array([0,0,0,1,1,2]), array([1,2,3,2,3,3]))
M = np.arange(16).reshape(4, 4)
M[iu] # array([1, 2, 3, 6, 7, 11]): 6 = 4*3/2 entries
```

`argsort` returns positions, not values. `triu_indices(N, k=1)` is a pair of index arrays that walk the entries above the diagonal in a fixed order; `D[iu]` pulls those $N(N - 1)/2$ entries out as a flat vector, and two matrices indexed with the same `iu` line up pair for pair.

2.7 Reordering rows and columns together

```
order = np.array([2, 0, 1])
M[np.ix_(order, order)] # rows AND columns in that order: a 3 x 3 block
M[order, order] # NOT that: picks M[2,2], M[0,0], M[1,1] -> 3 diagonal entries
```

Two index arrays side by side are read as *pairs* of coordinates. `np.ix_` builds the open grid instead, which is what “sort the matrix by category” means.

2.8 Diagonals, percentiles, and missing values

```
np.fill_diagonal(M, 0) # in place; returns None
np.diag(M) # the diagonal as a vector
np.percentile(v, [2, 98]) # two numbers: 2nd and 98th percentile of v
np.allclose(a, b) # equal up to floating-point roundoff?

B = np.array([[1., np.nan], [3., 4.], [5., 6.]])
np.isnan(B).sum() # 1: how many NaNs
np.isnan(B).any(axis=0) # array([False, True]): which COLUMNS have any
np.nanmean(B, axis=0) # array([3., 5.]): mean ignoring NaNs
np.where(np.isnan(B), 0., B) # replace NaNs by 0 (or by anything else)
```

A single nan spreads through everything it touches: a mean, a correlation, a distance. Count them before trusting any number downstream.

2.9 Loading the data: .npz files

```
arc = np.load('file.npz', allow_pickle=True) # an archive, like a read-only dict
arc.files # the keys, e.g. ['early', 'middle', 'late', 'image_names']
X = arc['late'] # one array out
X.shape, X.dtype # e.g. (120, 4096) float32
X = X.astype(np.float64) # float32 -> float64 before computing distances
```

The notebooks load everything for you and print a variable list at the bottom of the setup cell. Read that list; it tells you every name, shape and dtype you will use. The float64 cast matters: with float32 arithmetic the diagonal of a distance matrix comes out visibly above zero.

3 SciPy and scikit-learn: the calls the notebooks use

3.1 Distances: pdist and squareform

```
from scipy.spatial.distance import pdist, squareform
d = pdist(A, metric='euclidean') # CONDENSED: shape (n*(n-1)/2,), one value per pair
D = squareform(d) # SQUARE: shape (n, n), symmetric, zero diagonal
d = squareform(D, checks=False) # and back; checks=False tolerates roundoff asymmetry
```

metric is any of 'euclidean', 'cosine', 'correlation', and others. Functions disagree about which form they want: imshow and MDS need the square; linkage needs the condensed form. **When a call complains about your distance matrix, this is almost always why.**

3.2 The scikit-learn pattern

Every scikit-learn tool works the same way: put the settings in the constructor, hand the data to fit_transform, and read anything it learned from an attribute with a trailing underscore.

```
from sklearn.decomposition import PCA
pca = PCA(n_components=2)
Y = pca.fit_transform(A) # A is (3, 2) in -> Y is (3, 2) out
pca.explained_variance_ratio_ # (2,): fraction of variance per component
np.cumsum(pca.explained_variance_ratio_) # running total, ends at 1.0 here
```

np.argmax on a boolean array returns the position of the first True (positions start at 0): np.argmax(np.array([False, False, True, True])) is 2.

3.3 MDS with a precomputed dissimilarity matrix

```
from sklearn.manifold import MDS
mds = MDS(n_components=2, metric='precomputed', metric_mds=False,
          init='random', n_init=4, max_iter=400, random_state=0,
          normalized_stress='auto')
Y = mds.fit_transform(D) # D square (n, n) in -> Y (n, 2) out
mds.stress_ # how badly the 2-D map distorts D; lower is better
```

- metric='precomputed': D already holds dissimilarities; do not recompute them from D as if it were data.
- metric_mds=False: nonmetric MDS, which fits only the rank order of the dissimilarities. True fits their values.

- `random_state`: MDS starts from a random layout, so two runs differ unless you fix the seed. `n_init` runs several starts and keeps the best.
- These spellings need `scikit-learn` ≥ 1.8 ; the Part 2 setup cell upgrades Colab for you. Older versions spell it `dissimilarity='precomputed'`.

3.4 Other embeddings, same pattern

```
from sklearn.manifold import TSNE
Y = TSNE(n_components=2, perplexity=30, init='pca', random_state=0,
         max_iter=1000).fit_transform(X) # X (n, d) in -> (n, 2) out
```

3.5 Correlations

```
from scipy.stats import spearmanr, pointbiserialr
res = spearmanr([1, 2, 3, 4], [1, 3, 2, 4])
res.statistic # 0.8: the rank correlation rho
res.pvalue # exists; but if the entries are pairs of images they are not
           # independent, so do not lean on it
pointbiserialr([0, 0, 1, 1], [1.0, 2.0, 5.0, 6.0]).statistic # binary vs continuous
np.corrcoef(a, b)[0, 1] # Pearson r, one pair
```

Both inputs must be flat vectors of the same length. To correlate two matrices, index both with the same `np.triu_indices` pair first (§2.6).

3.6 Fitting a curve: `curve_fit`, `polyfit`, `polyval`

```
from scipy.optimize import curve_fit

def line(t, m, c): # first argument is the data; the rest are parameters
    return m * t + c

t = np.array([0., 1., 2., 3., 4.])
y = np.array([1., 3., 5., 7., 9.])
popt, pcov = curve_fit(line, t, y, p0=[1, 0], maxfev=10000)
popt # array([2., 1.]): the fitted m and c
line(t, *popt) # the fitted values; * unpacks popt into m, c

coef = np.polyfit(t, y, 1) # straight line, degree 1: array([2., 1.]) slope, intercept
np.polyval(coef, t) # the line evaluated at t
np.linspace(0, 4, 50) # 50 evenly spaced points, for drawing a smooth curve
```

- `p0` is the starting guess. Start near the right scale: if `y` runs 0–10, a parameter that multiplies the whole curve should start near 10, not 1.
- `maxfev` is the iteration budget. If the fit fails with “Optimal parameters not found”, raise it; if it still fails, say so in your answer. A fit that will not converge is a finding, not a bug.

- `popt` comes back in the order of the function's parameters after the first argument. Feed it back with `*popt`.

R^2 , when a notebook asks for it, is provided as a function in that notebook. Its definition is $1 - \sum_i (y_i - \hat{y}_i)^2 / \sum_i (y_i - \bar{y})^2$: 1 is a perfect fit, 0 is no better than predicting the mean, negative is worse than that. It is a ratio, so two R^2 values compare only when computed against the same target.

3.7 Hierarchical clustering

```
from scipy.cluster.hierarchy import linkage, fcluster, dendrogram
Z = linkage(pdist(A), method='average') # takes the CONDENSED form
Z = linkage(squareform(D, checks=False), method='single') # from a square matrix
groups = fcluster(Z, t=2, criterion='maxclust') # cut into 2 groups: one label per row
groups = fcluster(Z, t=0.5, criterion='distance') # or cut at a height
dendrogram(Z, no_labels=True) # draw the tree
```

3.8 Silhouette scores

```
from sklearn.metrics import silhouette_samples
sil = silhouette_samples(Y, labels) # Y: (n, 2) COORDINATES, not a distance matrix
# labels: one group code per row; returns (n,)
```

Near 1: the point sits well inside its own group. Near 0: on a boundary. Negative: closer to another group than to its own. `sil[labels == g].mean()` is group g 's score.

4 Sanity checks on a distance matrix

Run these every time you have made one, before anything downstream reads it:

```
D.shape # (n, n)?
np.allclose(D, D.T) # symmetric? (1e-16 off is roundoff, not a bug)
np.allclose(np.diag(D), 0) # zeros on the diagonal?
(D >= 0).all() # nonnegative?
np.isnan(D).sum() # 0?
off = D[np.triu_indices(len(D), k=1)] # the off-diagonal entries, each pair once
off.min(), off.max() # their range
```

If the diagonal is 10^{-6} rather than 0, you computed in `float32`, or your formula reaches zero only up to roundoff; `np.fill_diagonal(D, 0)` is the fix. If `np.abs(D - D.T).max()` is 10^{-17} , nothing is wrong. Cosine and correlation distances must lie in $[0, 2]$; if you see values outside that range, a normalisation is missing or a vector had zero length.

5 Plots as instruments

```
fig, ax = plt.subplots(2, 3, figsize=(12, 7)) # a 2 x 3 grid; ax is a (2, 3) array
ax[0, 1].imshow(D, vmin=lo, vmax=hi) # a matrix as an image
ax[1, 0].scatter(Y[:, 0], Y[:, 1], s=8) # a 2-D embedding
ax[0, 0].set_title('...'); ax[0, 0].set_xticks([]); ax[0, 0].set_yticks([])
plt.tight_layout(); plt.show()
```

When a matrix plot looks washed out, a few extreme values are eating the colour scale: set `vmin` and `vmax` from percentiles (§2.8) instead of letting matplotlib choose. To colour a scatter by category, loop over the categories and call `ax.scatter` once per group with a `label=`, then `ax.legend()`.

6 Common errors and what they mean

- `ValueError: operands could not be broadcast together with shapes (120,4096) (120,)`
A reduction lost an axis. Add `keepdims=True` or `[:, None]` (§2.4).
- `ValueError: shapes (120,4096) and (120,4096) not aligned`
Matrix product with inner dimensions that do not match. One side needs a `.T`.
- `IndexError: index 120 is out of bounds for axis 0 with size 120`
Indices start at 0; the last row is 119.
- `ValueError: The truth value of an array with more than one element is ambiguous and/or between arrays. Use & / | with parentheses.`
- `ValueError: Distance matrix 'X' must be symmetric`
`ValueError: The condensed distance matrix must contain only finite values`
linkage was handed a square matrix, or `squareform` one with roundoff asymmetry. Pass `squareform(D, checks=False)`.
- `TypeError: MDS.__init__() got an unexpected keyword argument 'metric_mds'`
scikit-learn older than 1.8. Re-run the setup cell.
- `RuntimeError: Optimal parameters not found`
`curve_fit` ran out of iterations. Raise `maxfev`, improve `p0`.
- `NameError: name 'D_human' is not defined`
A cell that defines it has not been run in this session, or was run before you fixed it. Run cells top to bottom.
- `KeyError: 'late'`
Wrong key on an npz archive; print `arc.files`.
- `TypeError: 'numpy.float64' object is not callable`
A name that used to be a function is now a number, usually `r2 = r2(...)` or a variable named `sum`. Rename and re-run the definition.
- `NotImplementedError: 1b: write your code here, then delete this line.`
The guard line at the bottom of a TODO cell. Delete it once your code is in.

- `nan`
, with no error at all. A zero-variance unit was divided by its standard deviation, or a `nan` in the input propagated. Count NaNs and zero-variance columns before the computation, not after.

7 Reading printed shapes

Printed	Read it as
<code>(120, 4096)</code>	a matrix: 120 stimuli, each a vector of 4,096 features
<code>(4096,)</code>	one vector of 4,096 numbers; the trailing comma makes it a 1-tuple
<code>(120, 1)</code>	a column: 120 rows of one number each; broadcasts against <code>(120, 4096)</code>
<code>(120,)</code>	120 numbers; does not broadcast against <code>(120, 4096)</code>
<code>(120, 120)</code>	a square matrix over 120 stimuli: a similarity or dissimilarity matrix
<code>(7140,)</code>	the upper triangle of a 120×120 matrix: $120 \cdot 119/2$ pairs
<code>(120, 2)</code>	120 points in the plane: an MDS, PCA or t-SNE embedding
<code>(1224, 200, 200, 3)</code>	1,224 images of 200 by 200 pixels with 3 colour channels
<code>(1224, 120000)</code>	the same images, each flattened to one vector ($200 \cdot 200 \cdot 3$)
<code>()</code>	a single number wrapped in an array; <code>float(x)</code> unwraps it

Write the shape you *expect* in a comment before you run the line, then compare. The notebooks' provided cells print shapes at every load for exactly this reason.

8 A debugging recipe

1. **Shape.** Print it. Is it what you expected, and the right way round?
2. **Type and range.** `X.dtype`, `X.min()`, `X.max()`. `uint8` and `float32` both cause quiet, plausible-looking wrongness.
3. **Missing values.** `np.isnan(X).sum()`.
4. **A case you can check by hand.** Run your code on a 3×2 array like `A` above and compare with a calculator.
5. **Plot it.** A wrong matrix usually looks wrong.
6. **Say what the line computes**, in one sentence, before fixing it. If you cannot, the problem is the idea, not the syntax, and the linear-algebra handout is the place to go.

Nothing on these pages answers a graded question. The notebooks' TODO cells name the functions they need; this handout says what those functions do.